# Reporte de seguridad — OWASP Top 10:2025

Financiamiento de Cuentas por Pagar. Banco Davivienda Salvadoreño · Banca Empresas. Revisión del 13/10/2026.

| Pieza | Carpeta | Versión revisada |
|---|---|---|
| API | `CXP_DAV_SERVER_REFACTOR` | Spring Boot 3.3.0, Java 21 |
| Portal SSO | `financiamientocuentasporpagar` | Spring Boot 3.4.4, Java 21 |
| Frontend | `CXP_DAV_CLIENT_REFACTOR` | React 19, Vite 6 |

## 1. Alcance y método

- **Revisión estática** del código fuente, la configuración (`application.properties`, `.env`, `SecurityConfig`), los scripts SQL (`db/prod`, `db/migration`) y los archivos versionados en git. No se ejecutaron pruebas de penetración contra un ambiente desplegado.
- **Análisis de dependencias:** `npm audit` para el frontend. Para los dos proyectos Java, la lista resuelta de dependencias de ejecución (102 en la API, 55 en el portal) se consultó contra la base pública OSV (osv.dev), que agrega los avisos de GitHub, NVD y los fabricantes.
- **Severidad:** Crítica, Alta, Media o Baja según la facilidad de explotación y el impacto financiero o sobre datos personales, no solo por la categoría de OWASP.
- Los secretos se muestran enmascarados.
- **Fuera de alcance:** el inicio de sesión por DUI (`POST /api/v1/auth/sso-login`), que es un acceso exclusivo del ambiente de desarrollo.

## 2. Resumen ejecutivo

El riesgo general es **crítico**. La llave que firma los JWT está en el código fuente y en el script de instalación de producción, lo que permite fabricar una sesión válida para cualquier usuario, incluido un operador bancario.

Con ese hallazgo corregido, el riesgo baja a **moderado**. La base del control de acceso es sólida:

- todo endpoint no listado se deniega;
- se valida que pagadores y proveedores solo operen sobre su propia entidad;
- las consultas son parametrizadas;
- el cupo y los documentos se bloquean durante las operaciones que los modifican.

En dependencias, el volumen de alertas es alto (6 en el frontend, 67 en la API y 58 en el portal), pero la mayoría no aplica al sistema o se limita a herramientas de desarrollo. El problema de fondo es que los dos proyectos Java usan versiones de Spring Boot que ya no reciben correcciones públicas.

## 3. Hallazgos en el código

| ID | OWASP | Severidad | Título | Ubicación |
|---|---|---|---|---|
| H1 | A04 | Crítica | Llave del JWT fija y versionada | `SystemParameterKey.java:36`, `V15__valor_por_defecto_jwt_secret.sql`, `instalacion_inicial.sql:899` |
| H2 | A09 | Alta | Tokens y secretos del SSO escritos en los logs | `SsoController.java:38,52,56`, `SsoServiceImpl.java:61,139` |
| H3 | A01 / A04 | Alta | La API de parámetros devuelve los secretos en claro | `ParameterDTOResponse.java`, `ParameterController.java:35` |
| H4 | A05 | Media | JSON armado concatenando el OTC | `SsoServiceImpl.java:96` |
| H5 | A03 | Media | Dependencias con vulnerabilidades publicadas (sección 4) | `pom.xml` de la API y del portal, `package.json` |
| H6 | A07 | Media | Sin límite de intentos y comparación de token no constante | `SsoServiceImpl.java:80`, endpoints públicos del SSO |
| H7 | A02 | Baja | Credenciales de base de datos por defecto | `application.properties:7-9` |
| H8 | A07 | Baja | JWT en `localStorage` y sin revocación al cerrar sesión | `AuthContext.jsx:164`, `api.js:17`, `SsoController.java:70` (portal) |
| H9 | A02 / A09 | Baja | Logs y compilados versionados en git | `CXP_DAV_SERVER_REFACTOR/logs/`, `financiamientocuentasporpagar/target/` |

Las rutas de archivos Java son relativas a `src/main/java/com/davivienda/factoraje/` en la API y a `src/main/java/com/davivienda/financiamiento/` en el portal.

### H1. Llave del JWT conocida — Crítica

- **Evidencia:** el valor por defecto (`G9U***`) está en el enum `SystemParameterKey`, en la migración V15 y en `instalacion_inicial.sql`. `SystemParameters` además lo usa como respaldo si el parámetro no existe.
- **Escenario:** con la llave cualquiera firma un JWT HS256 con el DUI de un operador como `sub`. El filtro solo comprueba la firma y que el usuario esté activo, así que el token es aceptado sin pasar por el SSO.
- **Recomendación:**
  - quitar el valor por defecto y fallar al arrancar si la llave no está configurada;
  - generar una llave por ambiente (`openssl rand -base64 32`) fuera del repositorio;
  - que el script de producción deje `CONFIGURAR`;
  - cambiar la llave en todo ambiente ya instalado.
- **Estado:** corregido en la versión V19. `JWT_SECRET` no tiene valor por defecto en el código, el script de instalación lo deja en `CONFIGURAR`, la actualización V19 revierte la llave por defecto donde siga puesta,

y la API no arranca sin una llave propia del ambiente. La llave anterior sigue en el historial de git y en V15, así que no debe reutilizarse.

### H2. Secretos en los logs — Alta

- **Evidencia:**
  - `SsoController` (API) escribe en el log el encabezado `Authorization`, que contiene el `BEARER_TOKEN` compartido con Davivienda, además del `sessionToken` y el JWT completo;
  - `SsoServiceImpl` escribe el OTC y el `sessionToken`.
- **Escenario:** quien tenga acceso a los logs puede suplantar el handoff o reutilizar un JWT vigente.
- **Recomendación:** quitar esos valores de los logs o enmascararlos, como ya hace el portal en `SsoController.java:46`.
- **Estado:** corregido. Los logs del SSO ya no incluyen el encabezado `Authorization`, el OTC, el `sessionToken` ni el JWT, y esos campos se excluyen del `toString()` de los DTO del SSO. Los logs generados antes del cambio pueden contenerlos, así que conviene depurarlos o restringir su acceso.

### H3. Secretos expuestos por la API de parámetros — Alta

- **Evidencia:** `ParameterDTOResponse.fromEntity` devuelve el valor sin filtrar. La marca `sensitive()` de `SystemParameterKey` solo evita escribir el valor en el log (`ParameterServiceImpl.java:67`).
- **Escenario:** un administrador del sistema, o quien robe su sesión, puede leer en el navegador `JWT_SECRET`, `MAILJET_API_SECRET` y los tokens del SSO.
- **Recomendación:** devolver los valores sensibles enmascarados y permitir solo escribirlos. Marcar `BEARER_TOKEN` y `PAY_BEARER_TOKEN` como sensibles.
- **Estado:** corregido. La API devuelve `********` en lugar del valor de `JWT_SECRET`, `MAILJET_API_KEY`, `MAILJET_API_SECRET`, `BEARER_TOKEN`, `PAY_BEARER_TOKEN` y `APP_CODE`, tanto al listar como al actualizar. Solo muestra `CONFIGURAR` cuando el secreto falta. La pantalla pide el valor nuevo en un campo de contraseña vacío, y la API rechaza que se guarde la máscara.

### H4. Inyección en el JSON enviado a Davivienda — Media

- **Evidencia:** `String jsonBody = "{\"otc\":\"" + otc + "\"}";`, con el OTC tal como llega en la petición.
- **Escenario:** un OTC con comillas altera el JSON que se envía al servicio externo de validación.
- **Recomendación:** serializar un DTO o un `Map` con el `RestTemplate` y validar el formato del OTC antes de enviarlo.

### H5. Dependencias vulnerables — Media

Ver el detalle en la sección 4.

### H6. Sin protección contra fuerza bruta — Media

- **Evidencia:** `handoff` y `activateSession` no tienen límite de intentos. El token compartido se compara con `String.equals`, que termina en el primer carácter distinto y por eso tarda distinto según cuántos

caracteres coincidan.

- **Recomendación:** limitar intentos en el F5 o en la API (por ejemplo, con Bucket4j) y comparar secretos con `MessageDigest.isEqual`.
- **Estado:** corregido en parte.
    - La comparación ya es de tiempo constante: el Bearer token y el `APP_CODE` se comparan con `MessageDigest.isEqual`.
    - El límite de intentos está pendiente. Antes de aplicarlo en la API hay que confirmar cómo reenvía el F5 la IP del cliente: si la API solo ve la IP del F5, todos los clientes comparten el mismo conteo y un atacante podría bloquear el acceso a todos. Mientras tanto, se recomienda limitar los intentos por IP en el F5.

### H7. Credenciales de base de datos por defecto — Baja

- **Evidencia:** `spring.datasource.password=${DB_PASSWORD:postgres}`. Si falta la variable, la API intenta conectar con `postgres/postgres`. En el ambiente se usa JNDI, lo que reduce el riesgo.
- **Recomendación:** dejar los valores por defecto solo en un perfil local.

### H8. Manejo del JWT en el navegador — Baja

- **Evidencia:** el token se guarda en `localStorage`, y al cerrar sesión no se invalida en el servidor: sigue siendo válido hasta que expira.
- **Mitigación actual:** no se encontró XSS. `react-markdown` no interpreta HTML y no hay `dangerouslySetInnerHTML`.
- **Recomendación:** mantener corta la duración del token (`JWT_EXPIRATION_MINUTES`). A mediano plazo, moverlo a una cookie `HttpOnly; Secure; SameSite` o llevar una lista de tokens revocados.
- **Estado:** corregido (versión V20).
    - El JWT viaja en la cookie `FIN_CXP_SESSION` (`HttpOnly; Secure; SameSite=Strict`, limitada a la ruta de la API). La arma la API; en el ingreso por SSO el portal reenvía el `Set-Cookie` de `activateSession` y redirige. JavaScript ya no puede leer el token y el navegador solo guarda su vigencia.
    - CSRF: `SameSite=Strict` evita que otro sitio envíe la cookie, y la API rechaza con 403 las peticiones de un origen que no esté en `CORS_ALLOWED_ORIGINS`, incluidos otros subdominios del banco.
    - Al cerrar sesión, por el botón o por inactividad, la API borra la cookie, guarda el momento en `users.sessions_revoked_at` y rechaza los tokens emitidos antes, en todas las pestañas y equipos del usuario.
    - Cada token dura lo que la inactividad permitida (`SESSION_IDLE_TIMEOUT_MINUTES`). El cliente lo renueva solo si hubo actividad, sin pasar de la duración máxima de la sesión (`JWT_EXPIRATION_MINUTES`).
    - Pendiente: una política CSP, que limitaría lo que un XSS podría hacer dentro de una sesión abierta.

### H9. Archivos que no deberían estar versionados — Baja

- **Evidencia:** en git están los logs de la API y el directorio `target/` del portal, incluidos `WEB-INF/lib/*.jar` y `application.properties`.
- **Contenido revisado:** los logs versionados no contienen tokens ni correos.
- **Recomendación:** agregar `logs/`, `*.log` y `target/` al `.gitignore` y retirarlos del índice con `git rm --cached`.

## 4. Dependencias

### 4.1 Frontend (`npm audit`)

| Paquete | Versión | Severidad | ¿Aplica en producción? |
|---|---|---|---|
| `vitest`, `tinypool`, `@vitest/mocker` | 3.2.7 / 1.1.1 | Crítica | No. Es el ejecutor de tests y no entra en el bundle. El riesgo se limita a equipos de desarrollo y CI. |
| `source-map-js` | 1.2.1 | Alta | No. Solo se usa al compilar (Tailwind y PostCSS). Se corrige con `npm audit fix`. |
| `react-router-dom`, `react-router` | 6.30.6 | Media | Parcial. El open redirect con `\` en `<Link>` y `useNavigate` aplica si se navega a URLs que controla el usuario. El fallo de SSR no aplica porque el frontend es una SPA. Se corrige en 7.18.x. |

Las versiones que `npm audit` propone como corrección (`vitest@2.0.5`, `react-router-dom@5.3.4`) son anteriores a las instaladas. La corrección correcta es subir a la versión más reciente.

### 4.2 API y portal (OSV)

| Proyecto | Dependencias | Vulnerabilidades | Spring Boot / Framework / Security |
|---|---|---|---|
| API | 102 | 67 | 3.3.0 / 6.1.8 / 6.3.0 |
| Portal | 55 | 58 | 3.4.4 / 6.2.5 / no usa Spring Security |

Según OSV, buena parte de las correcciones publicadas en 2026 solo existen en Spring Framework 7.0.x y Spring Boot 4.0.x. Las ramas 3.3 y 3.4 ya no reciben correcciones públicas.

**Aplican al sistema:**

| Componente | Vulnerabilidad | Por qué aplica | Corrección |
|---|---|---|---|
| Jackson 2.17.1 (API) / 2.18.3 (portal) | Varias de denegación de servicio con JSON malicioso (p. ej. CVE-2026-89407, CVE-2026-68497) | La API recibe JSON de los usuarios | 2.18.11 o superior |
| Spring MVC | CVE-2026-41853, contrabando de peticiones multipart | Hay carga de archivos | Spring Framework 7.0.8 |

| Componente | Vulnerabilidad | Por qué aplica | Corrección |
|---|---|---|---|
| Spring Security 6.3.0 | CVE-2026-22732, cabeceras de seguridad que no se escriben en algunos casos | La API depende de las cabeceras por defecto | Versión con soporte |
| Spring MVC 6.2.5 (portal) | CVE-2025-41242, path traversal en recursos estáticos | El portal sirve la SPA desde Spring | 6.2.10 o superior |
| Spring Web | CVE-2025-41234, descarga de archivos reflejada | La API devuelve PDF y Excel | 6.2.8 o superior |
| Driver PostgreSQL 42.7.5 | CVE-2025-49146 y CVE-2026-54291 (degradación de *channel binding*), CVE-2026-42198 (denegación de servicio por SCRAM) | Riesgo bajo: requiere un servidor de base de datos malicioso o en medio de la conexión | 42.7.12 |
| Apache HttpClient 5.4.2 (portal) | CVE-2025-27820, se desactiva la validación de dominio en TLS | El portal llama a la API por HTTPS | 5.4.3 o superior |
| `org.json` 20140107 (por `mailjet-client` 4.2.0) | CVE-2022-45688 y CVE-2023-5072, denegación de servicio | Riesgo bajo: solo parsea respuestas de Mailjet | Actualizar `mailjet-client` |
| Logback 1.5.6 / 1.5.18 | CVE-2025-11226 y otras | Riesgo bajo: requiere poder escribir el archivo de configuración | 1.5.34 |

**No aplican, porque el sistema no usa esas funciones:** XsltView, JSP, WebFlux, certificados X.509, Artemis, `DaoAuthenticationProvider`, BCrypt, Actuator (no está incluido), `CookieRequestCache` (la API no guarda sesión), `@AuthorizeReturnObject`, y el `Sort` de Spring Data a partir de la petición (los controladores fijan el orden).

**Fuera del alcance del código:**

- **Tomcat:** los dos proyectos se empaquetan como WAR con Tomcat en alcance `provided`. Sus vulnerabilidades dependen de la versión del Tomcat del servidor de aplicaciones, que debe confirmar infraestructura.
- **DevTools:** está declarado como `optional` y el empaquetado de Spring Boot lo excluye del WAR. Conviene confirmarlo en el artefacto final.

## 5. Controles revisados sin hallazgos

- **A01, control de acceso por datos:**
  - la regla por defecto es `denyAll`;
  - `CurrentUserService.requireOwnEntity` y `requireOwnMasterAgreement` se aplican en documentos, convenios, cargas y solicitudes de financiamiento;
  - la inactivación valida que el documento sea del pagador (`DocumentServiceImpl.java:229`).
- **A05, inyección:**
  - las consultas JPQL usan parámetros y no hay SQL nativo concatenado;
  - las plantillas de correo escapan HTML (`MailTemplateRenderer.escape`);

    - en el frontend no hay XSS.
- **A06, diseño:**
    - el costo del financiamiento se recalcula en el servidor;
    - documentos y cupo se bloquean con `SELECT ... FOR UPDATE;`
    - los índices únicos impiden el doble financiamiento;
    - un abono no puede superar el consumo del cupo.
- **A08, integridad:**
    - la carga valida extensión, tipo real del archivo, tamaño y macros, y Apache POI tiene límites contra archivos comprimidos maliciosos;
    - el `sessionToken` del SSO es de un solo uso y vence en 5 minutos.
- **A10, errores:** el manejador global responde con un mensaje genérico en errores 500, sin stacktrace.
- **Configuración:** los orígenes CORS se restringen por parámetro, y los archivos `.env` están excluidos de git.

## 6. Plan de remediación

| Prioridad | Acción | Hallazgos | Esfuerzo |
|---|---|---|---|
| 1 | Retirar la llave por defecto del JWT, exigirla al arrancar y cambiarla en los ambientes instalados | H1 | Bajo |
| 2 | Quitar tokens, OTC y JWT de los logs | H2 | Bajo |
| 3 | Enmascarar los parámetros sensibles en la API | H3 | Bajo |
| 4 | Armar el JSON del OTC con un serializador, comparar el token en tiempo constante y limitar intentos | H4, H6 | Bajo / Medio |
| 5 | Frontend: `npm audit fix` y actualizar `vitest` | H5 | Bajo |
| 6 | Subir ambos proyectos a la última versión de Spring Boot 3.5.x y fijar Jackson, el driver PostgreSQL y Logback en versiones corregidas | H5 | Medio |
| 7 | Planificar la migración a Spring Boot 4.0.x y React Router 7, con su propio ciclo de pruebas | H5 | Alto |
| 8 | Quitar de git los logs y `target/`, eliminar las credenciales por defecto | H7, H9 | Bajo |
| 9 | Evaluar mover el JWT a una cookie `HttpOnly` o agregar revocación | H8 | Medio |

Las prioridades 1 a 3 deberían quedar resueltas antes del pase a producción.